CS 473 (Spring 2025)
Homework 5 (due Mar 13 Thu 10am)

Instructions: As in previous homeworks.

Problem 5.1: In this problem, we will explore other simple families of hash functions and see how close they are to being universal. Let $U \geq m$.

(a) (35 pts) Let p be a random prime in $[m/2, m]$. Let b be a random number in $\{0, \ldots, p-1\}$. Define $h_p : \{0, \ldots, U-1\} \to \{0, \ldots, m-1\}$ by

$$h_p(x) = x \bmod p.$$

Prove that for any fixed $x, y \in \{0, \ldots, U-1\}$ with $x \neq y$, we have $\Pr_p[h_p(x) = h_p(y)] \leq O((\log U)/m)$.

(Thus, this hash function family is "almost" universal with an extra logarithmic factor.)

[*Hint*: By the prime number theorem, the number of primes in $[m/2, m]$ is $\Theta(m/\log m)$. Given a number $z \leq U$, how many prime factors in $[m/2, m]$ can z have?]

(b) (35 pts) Suppose $U = 2^w$ and $m = 2^\ell$. We can view numbers in $\{0, \ldots, U-1\}$ as binary vectors in $\{0,1\}^w$, and numbers in $\{0, \ldots, m-1\}$ as binary vectors in $\{0,1\}^\ell$.

Pick a random 0-1 matrix A of dimension $\ell \times w$ (each of its ℓw entries is an independently chosen random bit). Define $h_A : \{0,1\}^w \to \{0,1\}^\ell$ by

$$h_A(x) = (Ax) \bmod 2.$$

All arithmetic operations are done modulo 2; for example, addition is equivalent to exclusive-or.

Prove that for any fixed $x, y \in \{0,1\}^w$ with $x \neq y$, we have $\Pr_A[h_A(x) = h_A(y)] = 1/m$.

(Thus, this hash function family is universal, and its computation avoids integer division and requires only bitwise exclusive-or, which is cheaper. However, the evaluation time isn't $O(1)$.)

[*Hint*: suppose x and y differs at the i-th bit. Focus on choices over the i-th column of A...]

(c) (30 pts) Continuing part (b), let b be a random 0-1 vector of dimension ℓ. Define $h_{A,b} : \{0,1\}^w \to \{0,1\}^\ell$ by

$$h_{A,b}(x) = (Ax + b) \bmod 2.$$

Prove that for any fixed $x, y \in \{0,1\}^w$ with $x \neq y$ and for any fixed $s, t \in \{0,1\}^\ell$, we have $\Pr_{A,b}[(h_{A,b}(x) = s) \wedge (h_{A,b}(y) = t)] = 1/m^2$.

(Thus, this hash function family is 2-universal.)

[*Hint*: use (b).]

Solution:

(a) Let $z = |x - y|$ for distinct x, y. A collision occurs if and only if p divides z, i.e.,

$$h_p(x) = h_p(y) \quad \Leftrightarrow \quad p \mid z. \tag{1}$$

The number of primes in $[m/2, m]$ is given by the Prime Number Theorem as

$$\Theta(m/\log m). \tag{2}$$

If z has k prime factors in $[m/2, m]$, then

$$z \geq (m/2)^k. \tag{3}$$

Since $z \leq U$, we obtain

$$k \leq \log_{(m/2)} U. \tag{4}$$

Thus, z has at most $O(\log U)$ prime factors in $[m/2, m]$.
Since $U \geq m$, the probability of choosing a prime p that divides z is at most

$$\frac{O(\log U)}{\Theta(m/\log m)} = O\left(\frac{\log U}{m}\right). \tag{5}$$

(b) Let $z = x - y$, the collision condition is:

$$h_A(x) = h_A(y) \quad \Leftrightarrow \quad A(x - y) = 0 \quad \Leftrightarrow \quad Az = 0. \tag{6}$$

Since $x \neq y$, we have $z \neq 0$, meaning there exists at least one bit i such that $z_i = 1$.
Let $a_1, a_2, \ldots, a_w$ be the columns of A, and express:

$$Az = z_1 a_1 + z_2 a_2 + \cdots + z_w a_w. \tag{7}$$

Since $z_i = 1$ for at least one i, rewrite:

$$Az = a_i + \sum_{j \neq i} z_j a_j. \tag{8}$$

The elements of $a_1, a_2, \ldots, a_w$ are chosen independently and uniformly at random from $\{0, 1\}$, thus each element of $\sum_{j \neq i} z_j a_j$ is independent and uniform. So the probability that $Az = 0$ is simply the probability that each element of $a_i = 0 - \sum_{j \neq i} z_j a_j$, this occurs with probability $1/2^\ell$.
Thus, the probability of a collision is:

$$\Pr[Az = 0] = \frac{1}{2^\ell} = \frac{1}{m}. \tag{9}$$

(c) Let $z = x - y$,

$$(h_{A,b}(x) = s) \wedge (h_{A,b}(y) = t) \Rightarrow A(x - y) = s - t \Leftrightarrow Az = s - t \tag{10}$$

For fixed s and t, the probability that $Az = s - t$ is the probability that each element of $a_i = (s-t) - \sum_{j \neq i} z_j a_j$, this occurs with probability $1/2^\ell$:

$$\Pr_A[Az = s - t] = \frac{1}{m} \tag{11}$$

Given A s.t. $Az = s - t$,

$$(h_{A,b}(x) = s) \wedge (h_{A,b}(y) = t) \Leftrightarrow b = Ax - s \tag{12}$$

Since b is a random 0-1 vector, we have

$$\Pr_b[b = Ax - s | Az = s - t] = \frac{1}{m} \tag{13}$$

Combining 11 and 13, we obtain:

$$\begin{aligned} &\Pr_{A,b}[(h_{A,b}(x) = s) \wedge (h_{A,b}(y) = t)] \\ &= \Pr_A[Az = s - t] \Pr_b[b = Ax - s | Az = s - t] \\ &= \frac{1}{m^2} \end{aligned} \tag{14}$$

Problem 5.2: We are given a subset S of $\{1, \ldots, N\}$ and a number $k \leq N$. We want to compute a new set $f_k(S) = \{a_1 + \cdots + a_k : a_1, \ldots, a_k \text{ are } k \textit{ distinct} \text{ elements of } S\}$.

(Note that this is a little different from one of the midterm 1 practice problems, which is about computing the set $S + \cdots + S$ with k terms, where an element may be used more than once in the sum.)

(a) (50 pts) First consider a related problem: given ℓ disjoint sets $S_1, \ldots, S_\ell \subseteq \{1, \ldots, N\}$ and a number $k \leq \ell \leq N$, compute a new set $g_k(S_1, \ldots, S_\ell) = \{a_1 + \cdots + a_k : a_1, \ldots, a_k \text{ are elements chosen from } k \textit{ different} \text{ sets among } S_1, \ldots, S_\ell\}$.

Give a (deterministic) algorithm that solves this problem in $O(\ell^3 N \log N)$ time or better.

[Hint: use divide-and-conquer, like in the midterm 1 practice question. Recall that for $A, B \subseteq \{1, \ldots, M\}$, we can compute $A + B = \{a + b : a \in A,\ b \in B\}$ in $O(M \log M)$ time by convolution/FFT. It may be helpful to extend the problem to computing $g_k(S_1, \ldots, S_\ell)$ for all $k \leq \ell$ simultaneously.]

(b) (50 pts) Now design and analyze a Monte Carlo algorithm to compute $f_k(S)$. The running time should be of the form $O(k^c N \log^{c'} N)$ for some constants c, c', and the overall error probability should be at most $1/N$.

[Hint: partition S into ℓ disjoint sets randomly or by using a hash function, and then use part (a). How should we set ℓ?]

Solution:

(a) Split sets into two halves L and R, compute all $g_k(0 \le k \le \ell)$ using divide-and-conquer. For all possible $g_i(L)$ and $g_j(R)$ s.t. $i + j = k$, compute $g_i(L) + g_j(R)$ using FFT. Take the union of all such sets into $g_k(S_1, \ldots, S_\ell)$.

Algorithm 1

```
function DisjointSubsetSum(S)
    ℓ = length of S
    if ℓ = 0 then
        return φ
    L = S_1, S_2, ..., S_⌊ℓ/2⌋
    R = S_⌊ℓ/2⌋+1, ..., S_ℓ
    G_L = DisjointSubsetSum(L)
    G_R = DisjointSubsetSum(R)
    Initialize array G of length ℓ + 1, each element is φ
    for i = 0 to ⌊ℓ/2⌋ do
        for j = 0 to ⌊ℓ/2⌋ + 1 do
            if i + j ≤ ℓ then
                Compute g = g_i(L) + g_j(R) by taking convolution of G_L[i] and G_R[j]
                G[i + j] = g ∪ G[i + j]
    return G
S = [S_1, S_2, ..., S_ℓ]
return DisjointSubsetSum(S)[k]
```

Time complexity analysis:

Steps 1 and 2 split the problem into 2 subproblems and solve them recursively. In step 3, both halves have $O(\ell)$ sets, so there are $O(\ell^2)$ pairs of (i, j) such that $i + j = k$. Since the largest number in any set is ℓN and $\ell \le N$, each set contains at most $O(\ell N)$ elements, computing each $g_i(L) + g_j(R)$ using FFT takes $O(\ell N \log N)$ time. Thus the total time complexity is:

$$T(\ell, N) = 2T(\ell/2, N) + O(\ell^3 N \log N) = O(\ell^3 N \log N). \tag{15}$$

(b) Randomly partition S in to ℓ disjoint sets. For any specific set of k elements $(a_1, \ldots, a_k)$, the sum of the set s appears in $g_k(S)$ if each element is properly assigned to different disjoint subset. Since s can be computed by multiple sets, the possibility that $f_k(S)$ contains s is:

$$q = \Pr[s \in g_k(S)] \ge \prod_{i=0}^{k-1} \frac{\ell - i}{\ell}. \tag{16}$$

Let $\ell = k$, $q = \frac{k!}{k^k}$ is a constant for a particular k. Assume we randomly partition S for c times and compute $g_{k,i}(S)$ for the ith partition, the possibility that s does not appear in any $g_{k,i}(S)$ is:

$$\Pr\left[\bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] = (1 - q)^c \tag{17}$$

There are at most kN elements in $f_k(S)$, accroding to Union Bound, the possibility that the union of all $g_{k,i}(S)$ does not contain all possible elements is:

$$\Pr\left[\bigcup_{s\in f_x(S)}\bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] \leq kN \cdot \Pr\left[\bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] = kN(1-q)^c \tag{18}$$

Rewrite $kN(1-p)^c \leq 1/N$, we obtain,

$$c \geq \frac{2logN}{|log(1-q)|} \tag{19}$$

Algorithm 2

$c = \lceil 2logN \rceil$
for $i = 1$ to c **do**
    Randomly partition S into k subsets
    Compute $g_{k,i}(S)$ using algorithm in (a)
return $\bigcup_{i=1}^{c} g_{k,i}(S)$

Let $c = O(logN)$, overall error probability is at most $1/N$, total time complexity is $O(k^3 N log^2 N)$.

Problem 5.3: Consider the following geometric problem: given a set P of n points in 2D, with integer coordinates from $\{0, 1, \ldots, U-1\}$, find a *closest pair*, i.e., two points $p, q \in P$ ($p \neq q$) with the smallest Euclidean distance. Let $\delta(P)$ denote the distance of the closest pair.

We have seen an $O(n \log n)$-time divide-and-conquer algorithm from class. In this question, we give a different, faster randomized algorithm (which has the added advantage that it can be extended to higher dimensions).

(a) (35 pts) First give an $O(n)$-expected-time (Las Vegas) algorithm for the easier *decision problem*: given a value r, decide whether $\delta(P) < r$.

(Hints: Build a uniform grid where each cell is an $(r/2) \times (r/2)$ square. Use hashing. How many points can a grid cell have? For each grid cell, how many grid cells are of distance at most r?)

(b) (65 pts) Now, consider the following recursive Las Vegas algorithm to compute $\delta(P)$:

CLOSEST-PAIR(P):
1. if $|P| \leq 100$ then return answer by brute force
2. partition P into subsets $P_1, \ldots, P_{20}$ each with at most $\lceil n/20 \rceil$ points
3. let $S = \{(i, j) \mid 1 \leq i < j \leq 20\}$
4. $r = \infty$
5. for each $(i, j) \in S$ *in random order* do
6. if $\delta(P_i \cup P_j) < r$ then
7. $r =$ CLOSEST-PAIR($P_i \cup P_j$)
8. return r

Explain why the algorithm is always correct, and analyze its expected running time by solving a recurrence.

(Hints: Where is part (a) used? What is $|S|$? What is the expected number of times line 7 is performed, using facts from class?)

Solution:

(a) Use a uniform grid to efficiently check for point pairs within distance r. Divide the plane into a uniform grid where each cell is a square of size $(r/2) \times (r/2)$. If any cell contains two or more points, their distance is necessarily less than r. If a point has no neighbor in its own cell, it only needs to check points in adjacent cells (at most 24 cells since the side length of the square is $r/2$). Use a hash table to store the occupied grid cells, ensuring $O(1)$ insertion and lookup in expectation.

Algorithm 3

Initialize a hash table T
for each point $p = (x, y) \in P$ **do**
    $(v, w) = \left(\left\lfloor \frac{x}{r/2} \right\rfloor, \left\lfloor \frac{y}{r/2} \right\rfloor \right)$
    Transform and combine v and w into a 0-1 vector: $\{0,1\}^{2log_2 U}$, as the index of p.
    Store p in the hash table T:
    if another point already exists in the cell **then**
        return True
    else
        Check all 24 neighboring cells (a 5×5 grid centered at p). If any existing point in these cells is within distance r, **return** True.
return False

Time complexity analysis:

For each point, computing the grid index takes $O(1)$; Hash table operations (insertion and lookup) take expected $O(1)$; Checking adjacent cells involves at most a constant number of comparisons, also $O(1)$. Thus, for n points, the expected total runtime is $O(n)$.

(b) The algorithm correctly finds the closest pair distance $\delta(P)$ by employing a divide-and-conquer strategy with a randomized approach.

Base Case: If $|P| \leq 100$, the algorithm directly computes the closest pair using brute force, which is guaranteed to be correct.

Partitioning: The set P is divided into 20 subsets $P_1, P_2, \ldots, P_{20}$, each containing at most $\lceil n/20 \rceil$ points.

The algorithm considers all pairs of subsets from the set

$$S = \{(i, j) \mid 1 \leq i < j \leq 20\}.$$

Since there are 20 subsets, the number of pairs is:

$$|S| = \binom{20}{2} = 190.$$

The algorithm iterates over the pairs (i, j) in random order. If $\delta(P_i \cup P_j) < r$, it updates r with the result of a recursive call to ClosestPair$(P_i \cup P_j)$. The global closest pair distance $\delta(P)$ must exist in at least one of these subset pairs. Thus, as long as the correct pair (i^*, j^*) is considered, the algorithm will correctly compute $\delta(P)$.

The decision algorithm from part (a) is used to check whether $\delta(P_i \cup P_j) < r$ before making recursive calls.

Since the algorithm always considers a valid pair that contains the closest points, it will return the correct answer.

Time complexity analysis:

Let $T(n)$ denote the expected runtime of the algorithm. The partitioning step (line 2) takes $O(n)$ time. There are 190 pairs, and each pair is checked in constant time (except for potential recursive calls).

For each pair (i, j), the decision problem (from part (a)) runs in $O(n)$ expected time. Since the subset $P_i \cup P_j$ has at most $O(n/10)$ points, the decision step per pair runs in $O(n)$. However, not all 190 pairs will result in recursive calls. Since the pairs are traversed in a random order, the optimal pair (i^*, j^*) appears with equal probability at some position within the first k iterations. Once the correct minimum distance $\delta(P)$ is found, subsequent pairs will not trigger recursive calls. Thus the expected number of times line 7 (recursive call) executed is a constant.

Therefore, in expectation, the algorithm makes a recursive call on a subset of at most $O(n/10)$ points, a constant number of times.

The expected runtime satisfies the recurrence:

$$T(n) = O(n) + O(1) \cdot T(n/10).$$

Expanding the recurrence:

$$T(n) = O(n) + T(n/10).$$

Applying the recurrence iteratively:

$$T(n) = O(n) + O(n/10) + O(n/10^2) + O(n/10^3) + \dots.$$

This forms a geometric series:

$$O\left(n \sum_{k=0}^{\log_{10} n} \frac{1}{10^k}\right).$$

Since the sum of the geometric series converges to a constant, we get:

$$T(n) = O(n).$$